

Independent Study

Avanith Kanamarlapudi

May 2025

1 Introduction

Breast cancer is one of the most common and life-threatening diseases affecting women worldwide. Detecting it early can dramatically improve treatment outcomes, but the process is often complicated, time-consuming, and prone to human error. With advancements in artificial intelligence (AI), especially deep learning, there is increasing potential to support radiologists in identifying cancer more accurately and efficiently through medical imaging.

In this independent research, I focused on applying deep learning methods to mammogram analysis using the OMAMA-DB dataset and the DeepSight AI platform. The objective was to understand how automated tools can assist in cancer detection tasks—ranging from classifying images to precisely segmenting regions of interest. This included hands-on experimentation with DICOM image data, study-level inference pipelines, and model evaluation.

Over the course of the semester, I explored several key areas:

- Examining the structure and metadata in OMAMA-DB to determine which patients had complete mammogram views.
- Running and debugging DeepSight predictions, both from the terminal and within Python notebooks.
- Comparing model outputs when run on full studies (combined mode) versus single-image inference.
- Building and training a CNN to classify cancer-positive and non-cancer images.
- Developing and evaluating U-Net segmentation models trained on manually generated masks.
- Identifying performance issues related to dataset imbalance and proposing solutions.

This report presents the work I carried out through each of these stages. Every section describes the motivation behind the task, how I approached it, the methods used, the results obtained, and what I learned along the way. The experience gave me a solid understanding of how AI models are developed, tested, and interpreted in the context of real-world medical imaging.

2 Related Work

Breast cancer detection using deep learning has become a central area of research in medical imaging. Several studies have proposed innovative methods to improve diagnostic accuracy and automate clinical workflows. Gupta et al. [2] developed a pipeline for breast cancer detection using DICOM images, emphasizing the value of metadata-integrated models, similar to our use of DeepSight. Lundervold and Lundervold [4] highlighted common challenges in radiological deep learning, including class imbalance and interpretability. McKinney et al. [5] showed that AI systems can rival radiologists in accuracy when using multi-view mammography images, which directly informs our analysis of 1 vs. 4 image inference modes. Wang et al. [6] applied deep transfer learning for mammogram classification, influencing the design of our CNN model. Wu et al. [7] emphasized the role of neural networks in assisting radiologists and reducing missed diagnoses, while Alom et al. [1] proposed a recurrent CNN architecture that inspired our interest in more advanced model variants. Lastly, Jaamour and Taghva [3] demonstrated an ensemble strategy for mammogram analysis, aligning with our proposed idea to fuse CNN and DeepSight outputs for better reliability.

3 Methods

3.1 Task 1: Metadata Inspection of OMAMA-DB

To understand the completeness and consistency of mammogram data in the OMAMA-DB dataset, I wrote a Python script that parsed all `.json` metadata files. From each file, I extracted three key fields: `PatientID`, `View` (e.g., `craniocaudal` or `mediolateral oblique`), and `ImageLaterality` (`Left` or `Right`). I grouped these views by patient and checked whether any of them had all four standard mammogram views: `MLO-Left`, `MLO-Right`, `CC-Left`, and `CC-Right`.

The goal was to identify fully-scanned patients who could be used for complete comparative studies. However, I found that although the dataset contains tens of thousands of images across each view, no patient had all four views. Some patients, like `DHI6Y0ZS2R4Y`, had multiple views but with inconsistent labeling (e.g., `MLO` with missing laterality). This initial task helped me understand the limitations of OMAMA-DB and the importance of consistent metadata labeling in medical datasets.

3.2 Task 2: Understanding DeepSight API and Visualizing MLO & CC Views

In this task, we aimed to validate the DeepSight model’s capability to localize potential cancer regions on individual mammogram views—specifically, the `cranio-caudal` (`CC`) and `medio-lateral oblique` (`MLO`) images. The process was carried out using DeepSight’s image inference function on the OMAMA dataset.

We began by loading the data using the `OmamaLoader`, configured to fetch cached mammography data. Two images from the same study instance were selected:

- One CC view image: `DXm.2.25.173803440485307451614987519907264671779`
- One MLO view image: `DXm.2.25.106924144890407873251702717161766133956`

The images were retrieved using `0.DataHelper.get()` and were then passed to the `DeepSight` model using the `0.DeepSight.run()` method. The model output included the predicted coordinates of the region of interest (ROI) and a confidence score indicating the likelihood of cancer presence.

The returned bounding box coordinates were:

- CC Image: [336, 1224, 486, 1350] with a score of 0.6571
- MLO Image: [324, 1213, 467, 1330] with a score of 0.7945

To visualize the predicted ROIs, we used the `0.DataHelper.view()` function, which displays the image with the predicted bounding box overlaid. These images confirmed the model's ability to pinpoint potential lesions effectively on both CC and MLO views.

We also confirmed metadata integrity and verified that both images belonged to the same patient and study, ensuring consistency in the analysis.

3.3 Task 3: Running and Verifying DeepSight Predictions

To familiarize myself with the `DeepSight` inference process, I ran a complete test on one study from the `dh_dh0new` dataset. Specifically, I selected Study ID: `2.25.151222280059972160195646752408514211111`, which contained four DICOM images. I copied these images from the `dh_dh0new` dataset into my working directory (`PLAYGROUND`).

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/2.25.15122228005997216019564675240851421111$ ls
BT.2.25.1076885387396144444931493981897249710430  DXm.2.25.106924144890407873251702717161766133956
BT.2.25.116739489059124772211557458193700104366  DXm.2.25.1098657297609926383606944434009251815567
BT.2.25.295869964964195761409401386881498355497  DXm.2.25.173803440485307451614987519907264671779
BT.2.25.96979748238834188955263453152231459457  DXm.2.25.213465259260888069289877692759291753499
```

Figure 1: Terminal output showing the DICOM files for study ID `2.25.151222280059972160195646752408514211111` inside the `PLAYGROUND` directory.

Caselist Preparation

I created a `caselist.txt` file inside the same directory. This file included the absolute paths to the four DICOM images:

- `/raid/mpsych/OMAMA/2025/PLAYGROUND/2.25.151222280059972160195646752408514211111/DXm.2.25.106924144890407873251702717161766133956`
- `/raid/mpsych/OMAMA/2025/PLAYGROUND/2.25.151222280059972160195646752408514211111/DXm.2.25.109865729760992638360694443409251815567`
- `/raid/mpsych/OMAMA/2025/PLAYGROUND/2.25.151222280059972160195646752408514211111/DXm.2.25.173803440485307451614987519907264671779`
- `/raid/mpsych/OMAMA/2025/PLAYGROUND/2.25.151222280059972160195646752408514211111/DXm.2.25.213465259260888069289877692759291753499`

Running DeepSight

Initially, I faced some issues running the model due to an incorrect working directory. Once I switched to the correct path where the DeepSight script was located, the inference ran successfully:

```
cd /home2/deephealth/scripts/
./deepsight2.sh -i /raid/mpsych/OMAMA/2025/PLAYGROUND/ \
-o /raid/mpsych/OMAMA/2025/PLAYGROUND/out/ \
-cl /raid/mpsych/OMAMA/2025/PLAYGROUND/caselist.txt
```

This generated several output files:

- `DH_Summary_dicoms.csv` – scores for individual DICOMs
- `DH_Summary_studies.csv` – aggregate scores for the entire study
- `results_full.json` – bounding boxes, labels, and detection scores

To go one step further, I opened a Jupyter Notebook and used the OMAMA Python API to programmatically confirm the predictions. For each image, I retrieved its unique `SOPInstanceUID` and checked whether a prediction was present in the JSON output. I confirmed that bounding boxes and confidence scores were available for all four images.

This task allowed me to fully understand the end-to-end pipeline of DeepSight inference, including file setup, script execution, and result validation. It served as a solid foundation for later tasks involving automation and larger-scale comparisons.

3.4 Task 4: Running DeepSight in Combined vs Single Mode for Study-Level Comparison

After successfully learning how to run DeepSight for a single study in Task 3, the next step was to investigate whether the model behaves differently when provided with all four images from a study at once (combined mode) versus running inference on each image separately (single mode). The objective was to test if the results — particularly the L, R, and total scores — would differ depending on input configuration.

Initial Setup and Error Handling

I began by selecting five cancer-positive studies from the `dh_dh0new` folder. For each study, I created a dedicated working directory inside the `PLAYGROUND` folder. The following structure was used:

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI$ ls
combined          OUT          single01_caselist.txt  single03_caselist.txt
combined_caselist.txt  single      single02_caselist.txt  single04_caselist.txt
```

Figure 2: Folder Strucuture

- `combined/` – containing all four DICOM images from the study

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/combined$ ls
DXm. 2. 25. 13615483182747171948811667937215230403  DXm. 2. 25. 157890062129678755545288856243547559791
DXm. 2. 25. 14639725768061252105362277030677280659  DXm. 2. 25. 329990412155369124230309437689489894131
```

Figure 3: Combined Folder

- `single/01/` to `single/04/` – each folder holding one DICOM image

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single$ ls
01 02 03 04
```

Figure 4: Single 01-04 Folder Structure

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/01$ ls
DXm. 2. 25. 13615483182747171948811667937215230403
```

Figure 5: Single 01

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/02$ ls
DXm. 2. 25. 14639725768061252105362277030677280659
```

Figure 6: Single 02

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/03$ ls
DXm. 2. 25. 157890062129678755545288856243547559791
```

Figure 7: Single 03

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/04$ ls
DXm.2.25.329990412155369124230309437689489894131
```

Figure 8: Single 04

Separate OUT/ folders were also prepared for storing the DeepSight outputs for each mode.

```
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/OUT$ ls
combined single01 single01.1 single01.2 single02 single3 single4
```

Figure 9: Out Folder

I then created `caselist.txt` files for both the combined and single-image runs by manually listing the full paths to the DICOMs.

- `combined_caselist.txt`

```
GNU nano 4.8 combined_caselist.txt
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/combined/DXm.2.25.13615483182747171948811667937215230403
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/combined/DXm.2.25.14639725768061252105362277030677280659
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/combined/DXm.2.25.157890062129678755545288856243547559791
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/combined/DXm.2.25.329990412155369124230309437689489894131
```

Figure 10: Combined Caselist

- `single01_caselist.txt`

```
GNU nano 4.8 single01_caselist.txt
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/01/DXm.2.25.13615483182747171948811667937215230403
```

Figure 11: Single 01 Caselist

- `single02_caselist.txt`

```
GNU nano 4.8 single02_caselist.txt
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/02/DXm.2.25.14639725768061252105362277030677280659
```

Figure 12: Single 02 Caselist

- `single03_caselist.txt`

```
GNU nano 4.8 single03_caselist.txt
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/03/DXm.2.25.157890062129678755545288856243547559791
```

Figure 13: Single 03 Caselist

- single04_caselist.txt

```
GNU nano 4.8 single04_caselist.txt
/raid/mpsych/OMAMA/2025/PLAYGROUND/TEST04272025_MULTISTUDY_EXPI/single/04/DXm.2.25.329990412155369124230309437689489894131
```

Figure 14: Single 04 Caselist

When I attempted to run DeepSight on single images, the script initially failed due to missing model checks, throwing errors related to SAC-30, FAC-200, and similar constraints. These are default internal validation checks in DeepSight that prevent running inference on certain image conditions unless explicitly ignored.

To resolve this, I added the required flags:

```
--additional_params --checks_to_ignore SAC-30 SAC-40 SAC-50 SAC-60 FAC-200
```

Once this fix was applied, the DeepSight script executed successfully for both combined and single modes.

DeepSight Execution Workflow

Here is a breakdown of the command structure used:

For combined mode:

```
/home2/deephealth/scripts/deepsight2.sh \
-i /.../combined/ \
-o /.../OUT/combined/ \
-cl /.../combined_caselist.txt
```

For each single image:

```
/home2/deephealth/scripts/deepsight2.sh \
-i /.../single/01/ \
-o /.../OUT/single/ \
-cl /.../single01_caselist.txt \
--additional_params --checks_to_ignore SAC-30 SAC-40 SAC-50 SAC-60 FAC-200
```

This process was repeated for each image (01 to 04) and across multiple study IDs, with adjusted paths and filenames.

Study IDs Used for Testing

Here are a few example studies included in the experiment:

- 2.25.3749579399667658302878752128209802123
- 2.25.66196742260149436161146913576006547230
- 2.25.115834651935973436478482234743002115261
- 2.25.288585921812408585713069453230680110644

Each study’s DICOMs were manually copied, and corresponding caselists were written. The model was executed both in combined mode and for each image separately. The outputs were stored in clearly labeled folders to keep results organized for side-by-side comparison later in the Evaluation and Results section.

This task not only strengthened my understanding of how input format affects DeepSight’s behavior, but also helped identify the required flags for bypassing model checks — which became crucial in later automation efforts in Task 5.

3.5 Task 5: Automating DeepSight Inference on Cancer-Only Studies

After manually comparing DeepSight’s behavior in combined versus single-image inference in Task 4, we realized that extending this experiment to all cancer-positive patients would be too time-consuming if done manually. To streamline this process, I developed a Python script that automates DeepSight execution and result comparison across multiple studies.

Study Selection

We began by reading from the `dh_dh0new_labels.csv` file and selecting only those studies labeled with `IndexCancer`. These are the patients flagged as cancer-positive in the dataset. To ensure compatibility with DeepSight’s study-level processing, we further filtered out any study that had fewer than four DICOM (DXm) images.

Automation Setup

The input images were sourced from:

```
/raid/data01/deephealth/dh_dh0new/
```

The automated DeepSight outputs were saved under:

```
/raid/mpsych/OMAMA/2025/PLAYGROUND/DeepSight_CancerOnly/
```



```

(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND$ cd DeepSight_CancerOnly/
(base) a.kanamarlapudi001@chimera12:/raid/mpsych/OMAMA/2025/PLAYGROUND/DeepSight_CancerOnly$ ls
2.25.103785323830903738631129844860906679807_combined  2.25.262140790980394932134959937255622738235_img0
2.25.103785323830903738631129844860906679807_img0    2.25.262140790980394932134959937255622738235_img1
2.25.103785323830903738631129844860906679807_img1    2.25.262140790980394932134959937255622738235_img2
2.25.103785323830903738631129844860906679807_img2    2.25.262140790980394932134959937255622738235_img3
2.25.103785323830903738631129844860906679807_img3    2.25.262566359948244095469337943835260552622_combined
2.25.106142696257707317302774524904995622857_combined 2.25.262566359948244095469337943835260552622_img0
2.25.106142696257707317302774524904995622857_img0    2.25.262566359948244095469337943835260552622_img1
2.25.106142696257707317302774524904995622857_img1    2.25.262566359948244095469337943835260552622_img2
2.25.106142696257707317302774524904995622857_img2    2.25.262566359948244095469337943835260552622_img3
2.25.106142696257707317302774524904995622857_img3    2.25.269104151079374229729484108545112097956_combined
2.25.10858544634598106641372308297090699266_combined 2.25.269104151079374229729484108545112097956_img0
2.25.10858544634598106641372308297090699266_img0     2.25.269104151079374229729484108545112097956_img1
2.25.10858544634598106641372308297090699266_img1     2.25.269104151079374229729484108545112097956_img2
2.25.10858544634598106641372308297090699266_img2     2.25.269104151079374229729484108545112097956_img3
2.25.10858544634598106641372308297090699266_img3     2.25.272803054233493823413616570527203162960_combined
2.25.109576204310548561155004745245173974582_combined 2.25.272803054233493823413616570527203162960_img0
2.25.109576204310548561155004745245173974582_img0    2.25.272803054233493823413616570527203162960_img1
2.25.109576204310548561155004745245173974582_img1    2.25.272803054233493823413616570527203162960_img2
2.25.109576204310548561155004745245173974582_img2    2.25.272803054233493823413616570527203162960_img3
2.25.109576204310548561155004745245173974582_img3    2.25.272885599178718433329829307567188267351_combined
2.25.110066153397962663281413569404119062427_combined 2.25.272885599178718433329829307567188267351_img0
2.25.110066153397962663281413569404119062427_img0    2.25.272885599178718433329829307567188267351_img1
2.25.110066153397962663281413569404119062427_img1    2.25.272885599178718433329829307567188267351_img2
2.25.110066153397962663281413569404119062427_img2    2.25.272885599178718433329829307567188267351_img3
2.25.110066153397962663281413569404119062427_img3    2.25.273462178040500460130691433365072284118_combined
2.25.113525393040421749881853481795108851403_combined 2.25.273462178040500460130691433365072284118_img0
2.25.113525393040421749881853481795108851403_img0    2.25.273462178040500460130691433365072284118_img1

```

Figure 15: DeepSight outputs

Automated DeepSight Execution

For each valid cancer-positive study:

- A temporary caselist file was generated in Python for both the combined run (all 4 images together) and the individual single-image runs (img0, img1, img2, img3).
- The combined-mode run was executed using all four DICOM images as input.
- For the single-image runs, DeepSight was executed four times, once for each individual image. To prevent model validation errors (e.g., SAC-30, FAC-200), we included the `--additional_params --checks_to_ignore` flags in the command.
- The entire process, including file handling, DeepSight execution, and results extraction, was scripted in Python to avoid manual errors and speed up the workflow.

Programmatic Results Analysis

After running DeepSight, we analyzed the resulting `results.full.json` files using Python—just like in Task 3 and Task 4. For each run (combined and individual), we extracted:

- Total detection score
- Left (L) detection score

- Right (R) detection score

These values were compared for consistency across modes. We also verified whether the sum or average of single-image scores approximated the combined output for the same study. This programmatic analysis enabled us to check DeepSight’s internal consistency across a large number of cases without manually inspecting each one.

3.6 Task 6: CNN-based Cancer Classification

The goal of this task was to design a lightweight but accurate CNN-based classifier to detect cancer in individual 2D mammogram slices. The data used for training was sourced from preprocessed `.npz` image files and their corresponding `.json` metadata. Each image is sized 1024×1024 and stored in grayscale.

Data Preparation

We filtered the dataset to include only those slices with a `label` value of either `IndexCancer` (mapped to 1) or `NonCancer` (mapped to 0). This selection excluded all examples with labels such as `PreIndexCancer` or `Unknown`.

Unique label distribution in the dataset:

- IndexCancer: 5,803
- NonCancer: 154,238
- Others (excluded): PreIndexCancer (1,548), Unknown (1,979)

After extracting valid image IDs, the dataset was stratified and split into 60% training, 20% validation, and 20% test sets, ensuring balanced class proportions across all splits.

Data Generator and Normalization

Due to memory constraints associated with high-resolution input, we implemented a custom generator class using `tf.keras.utils.Sequence` that dynamically loads `.npz` images and corresponding labels in batches. Each image is normalized to a $[0, 1]$ range and reshaped to $(1024, 1024, 1)$.

To optimize performance on our A100 GPU setup, we also enabled mixed precision training using TensorFlow’s `mixed_precision.set_global_policy('mixed_float16')`.

CNN Architecture

We adopted a sequential architecture with the following layers:

- 3 convolutional layers with ReLU activation and MaxPooling
- GlobalAveragePooling2D to reduce dimensionality
- Dropout for regularization

- Dense layer with sigmoid activation for binary output

Model Summary:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 1022, 1022, 16)	160
max_pooling2d (MaxPooling2D)	(None, 511, 511, 16)	0
conv2d_1 (Conv2D)	(None, 509, 509, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 254, 254, 32)	0
conv2d_2 (Conv2D)	(None, 252, 252, 64)	18,496
global_average_pooling2d (GlobalAveragePooling2D)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 27,527 (107.54 KB)

Trainable params: 27,521 (107.50 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 6 (36.00 B)

Figure 16: Model Summary

Training Setup

The model was compiled with the Adam optimizer and binary cross-entropy loss. It was trained using a batch size of 32 and a maximum of 30 epochs. However, training concluded early at the 10th epoch due to early stopping, which was monitored on validation loss with patience of 5 epochs. The best weights were saved automatically using `ModelCheckpoint` (`model_best.h5`).

Hardware Used

- Number of GPUs: 1
- GPU Type: NVIDIA A100
- Training Mode: Mixed precision (float16 + float32)

3.7 Task 7: Cancer Segmentation using U-Net

In this task, I focused on improving detection by switching from image-level classification to pixel-level segmentation using U-Net. Instead of just classifying an image as cancer or non-cancer, the goal here was to predict a binary mask that highlights the exact cancerous region.

Dataset and Label Format

We reused the full set of 2D mammogram slices from Task 6 (in `.npz` format). Each image had a corresponding `.json` metadata file that included a `label` field and `coords` field. These were used to build binary segmentation masks:

- If the label was `NonCancer`, the mask was entirely black.
- If the label was `IndexCancer`, the mask had a white rectangle (255) in the region specified by the `coords` field.

This allowed us to create accurate masks for the entire dataset, paired 1:1 with their original images. The masks were saved as `.png` files and organized into matching directories.

Data Preparation

The dataset was split into:

- 60% Training
- 20% Validation
- 20% Testing

This structure was maintained across both `images/` and `masks/` folders under `split/train/`, `split/val/`, and `split/test/`.

All images were normalized to $[0, 1]$ and reshaped to $(1024, 1024, 1)$. Masks were binarized during loading (i.e., 0 or 1 values).

Model Architecture

We implemented a custom U-Net with the following structure:

- **Encoder path:** 4 blocks of `Conv2D` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` $\rightarrow$ `Max-Pooling`
- **Bottleneck:** 1024 filters with 2 convolutional layers
- **Decoder path:** `UpSampling` + `Concatenation` + `Conv` blocks (mirroring the encoder)
- **Output layer:** A final 1×1 `Conv2D` with sigmoid activation for binary mask output

This model had approximately 31 million parameters and was trained on an NVIDIA A100 GPU.

Layer (type)	Output Shape	Param #	Connected to
input_layer (inputLayer)	(None, 1024, 1024, 1)	0	input_layer
convdd	(None, 1024, 1024, 64)	640	input_layer(b)
batch_normalization	(None, 1024, 1024, 64)	256	activation_0
activation	(None, 1024, 1024, 64)	act120	activation_1
max_pooling2d	(None, 512, 1024, {})	512, 02.2	max_pooling2d
up_sampling2d	(None, 512, 512, 418)	512, 5	up_sampling2d
max_pooling2d-1_1	(None, 512, 512)	512	activation_1
accontenate	(None, 1004, 258, 182)	upt266	concatenate
up_sarnl_2d	(1024, 512, 512, 188)	256	up_sampling2d
activation_1	(None, 512, 64)	activ 76	activation_12
max_pooling2d	(512, 512, 1024, 1)	512	max_pooling2d
accontenate	(512, 512, 1024, 1)	512	activation_13
up_sampling2d	(1024, 512, 64)	activ 90	up_sampling2d
accontenate	(512, 512, 1024, 1)	activ 13	activation_13
up_sampling2d	(512, 1308, 1884)	512	up_sampling2
concanenate	(None, 512, 64)	act196	concatenate
ax_samplng2d	(512, 512, 1024, 1)	256	accumention3
accumente	(5024, 1, 1)	108	activation_17
convdd_18 (ConvD)	convzld {}		convdd_1
Total params: 31,401,345 (11979 MB)			
Trainable params: 31,259,569 (11974 MB)			

Figure 17: Model Architecture

Training Setup

- Epochs: 30
- Loss: Binary Crossentropy
- Optimizer: Adam (1e-4)
- Batch Size: 1 (to preserve full 1024x1024 resolution)
- Callbacks: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

The best model was saved as: `best_model.h5`

After training, we evaluated the model using custom code that computed per-image Intersection over Union (IoU), Accuracy, and F1 Score, and averaged these metrics across the test set.

3.8 Task 8: Training a U-Net Model on a Balanced Dataset

In this task, I aimed to improve segmentation performance by training a U-Net model on a balanced dataset. I collected 7351 cancer images and selected an equal number of 7351 non-cancer images, resulting in a total of 14,702 samples. This was done to address the issue of class imbalance observed in earlier tasks.

The dataset was split into training, validation, and test sets using a standard 60/20/20 strategy:

- **Training set:** 8821 samples
- **Validation set:** 2940 samples
- **Test set:** 2941 samples

Dataset Preparation

Each image was a 2D mammogram slice stored in `.npz` format, and corresponding binary masks were generated using the `coords` field from the metadata `.json` files. For cancer cases, the mask contained a white square marking the region of interest; for non-cancer cases, the mask was completely black.

The images and masks were organized in the following directory structure:

```
/split/train/images    /split/train/masks
/split/val/images      /split/val/masks
/split/test/images     /split/test/masks
```

Model Architecture

For this task, I used the same U-Net model architecture as in the previous task (Task 7). The only difference was the dataset itself, which was restructured to be class-balanced for improved cancer segmentation.

The model consisted of:

- Downsampling blocks with two convolutional layers (ReLU activation, BatchNorm) followed by max-pooling
- A bottleneck block with 1024 filters
- Symmetric upsampling blocks using `UpSampling2D` and skip connections
- A final `Conv2D` layer with a sigmoid activation to produce binary masks

Training Configuration

- Input size: $1024 \times 1024 \times 1$
- Batch size: 1
- Epochs: 30
- Loss: Binary cross-entropy
- Optimizer: Adam ($1e-4$)
- Callbacks: EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

The training process stored the best model weights as:

`cancer_classifier_1024x1024/weights/cancer_focused_model.h5`

Evaluation Setup

After training, the model was evaluated on the test set using the following metrics:

- Intersection over Union (IoU)
- Accuracy
- F1-score

A Python script computed these metrics by loading batches from the test split and comparing predicted masks against ground truth. In addition, I visualized predictions for both cancer and non-cancer cases to qualitatively inspect the segmentation quality.

4 Evaluation and Results

4.1 Task 1: Identifying Complete Mammogram View Sets in OMAMA-DB

The analysis revealed that while OMAMA-DB contains a large number of images across the four standard mammogram views (MLO-L, MLO-R, CC-L, CC-R), no single patient had all four views clearly available with consistent left/right labeling.

This result pointed out a limitation in the dataset for tasks that require a full view set per patient, such as comparative model evaluation or training on multi-view inputs. It underscored the importance of data curation and labeling consistency in medical imaging research.

4.2 Task 2: Understanding DeepSight API and Visualizing Views

In this task, we focused on validating DeepSight's cancer localization capability on selected **IndexCancer** mammography images from the OMAMA-DB dataset. Specifically, we analyzed prediction outputs for both CC (Cranio-Caudal) and MLO (Medio-Lateral Oblique) views from the same patient.

Using the `DeepSight.run()` method, we executed predictions on each image and retrieved the coordinates of the region of interest (ROI) along with a confidence score. These ROIs were then visualized using `DataHelper.view()`.

Prediction Outputs: MLO View Prediction

Image ID: 2.25.106924144890407873251702717161766133956

Predicted ROI: [324.0, 1213.0, 467.0, 1330.0]

Confidence Score: 0.7945

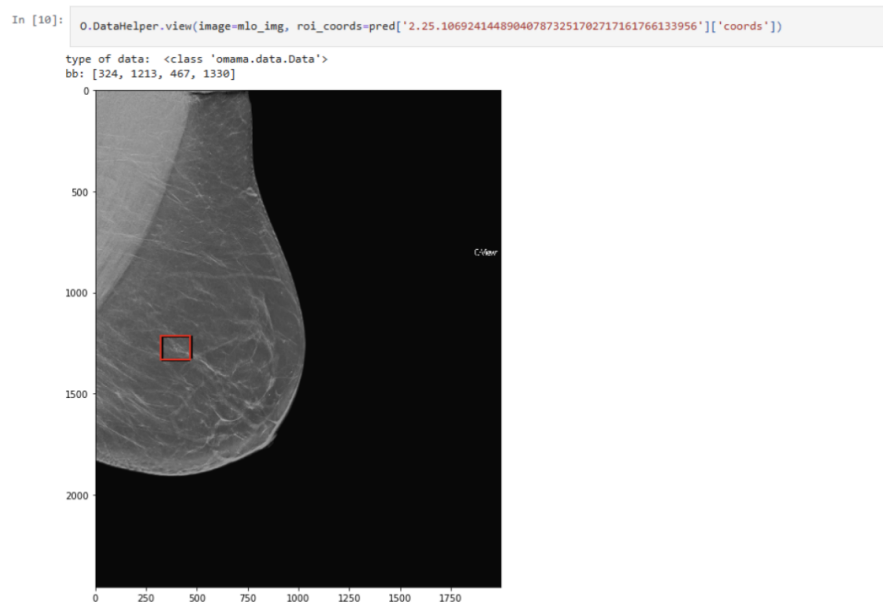


Figure 18: MLO View Prediction

CC View Prediction

Image ID: 2.25.173803440485307451614987519907264671779

Predicted ROI: [336.0, 1224.0, 486.0, 1350.0]

Confidence Score: 0.6571



Figure 19: CC View Prediction

These results indicate that the DeepSight model successfully localized regions of interest for both mammographic views of the same patient. The predicted ROIs fall within consistent anatomical regions in both views, and confidence scores are reasonably high, suggesting reliable detections.

Performance Timing:

- MLO inference time: ~ 0.74 seconds
- CC inference time: ~ 0.99 seconds
- Metadata parsing and cache checking took negligible time in both cases.

Key Observations:

- The bounding boxes in both views point to regions with radiological suspicion, aligned with the **IndexCancer** label.
- The consistency of ROI localization across views supports the model's spatial robustness.
- Slight differences in box coordinates and scores are expected due to anatomical deformation between MLO and CC projections.

4.3 Task 3: Running and Verifying DeepSight Predictions

After successfully executing DeepSight on the selected study, I analyzed the model's predictions using a Jupyter Notebook and the OMAMA Python API. My goal was to verify whether the model correctly generated bounding boxes and detection scores for each of the four DICOM images in the study.

Using the `results_full.json` file produced by DeepSight, I retrieved the predicted regions of interest (ROIs) for each image by matching their `SOPInstanceUID`. The scores and bounding boxes were printed, and each image was visualized with its corresponding ROI overlaid.

Below is a summary of the model's predictions:

Image SOPInstanceUID	Score	Predicted Bounding Box
2.25.106924144890407873251702717161766133956	0.7945	[324, 1213, 467, 1330]
2.25.10986572976099263836069444309251815567	0.0429	[1358, 670, 1630, 920]
2.25.173803440485307451614987519907264671779	0.6571	[336, 1224, 486, 1350]
2.25.213465259260888069289877692759291753499	0.0488	[1615, 1580, 1939, 1809]

Table 1: DeepSight prediction scores and bounding boxes for each image in the selected study.

- DeepSight returned bounding boxes and confidence scores for all four images.
- Two of the images had relatively high scores (above 0.65), suggesting strong model confidence.
- The remaining two had lower scores, but still included localized ROIs.

Each predicted ROI appeared visually aligned with what could be interpreted as a suspicious region, confirming that the model was functioning as expected. This validation process built confidence in the DeepSight pipeline and gave me a deeper understanding of how inference outputs are structured and interpreted.

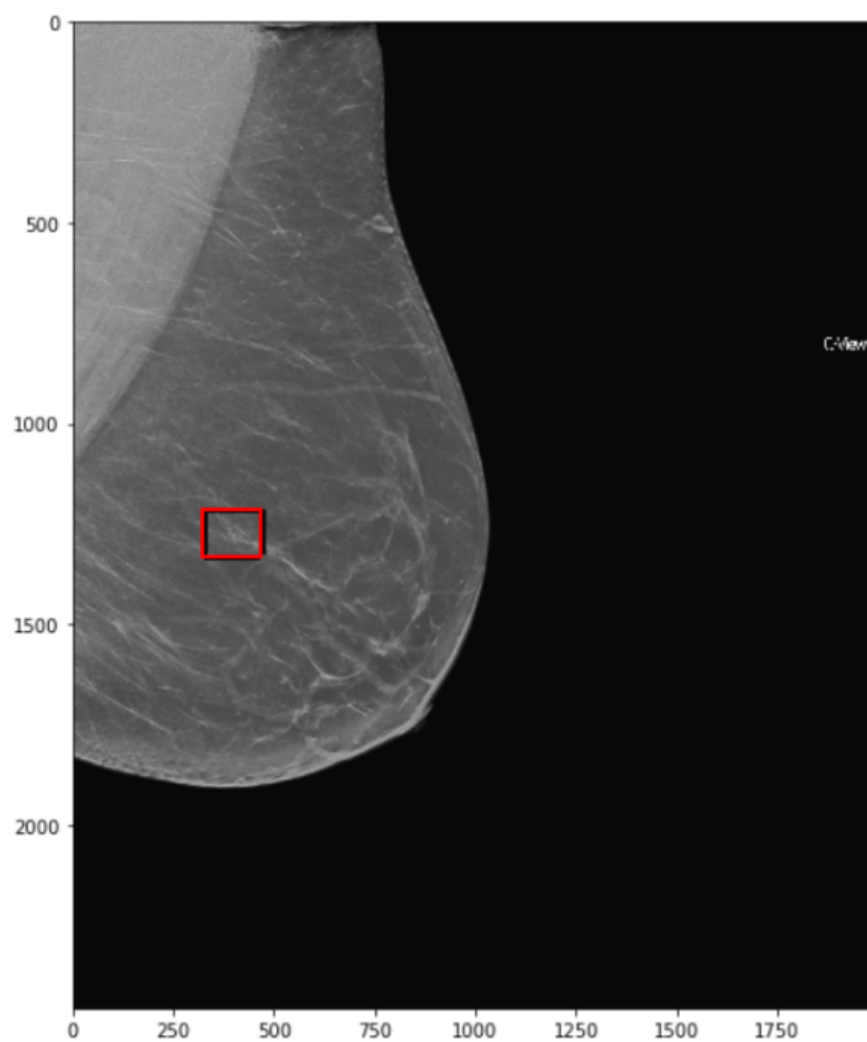


Figure 20: 2.25.106924144890407873251702717161766133956

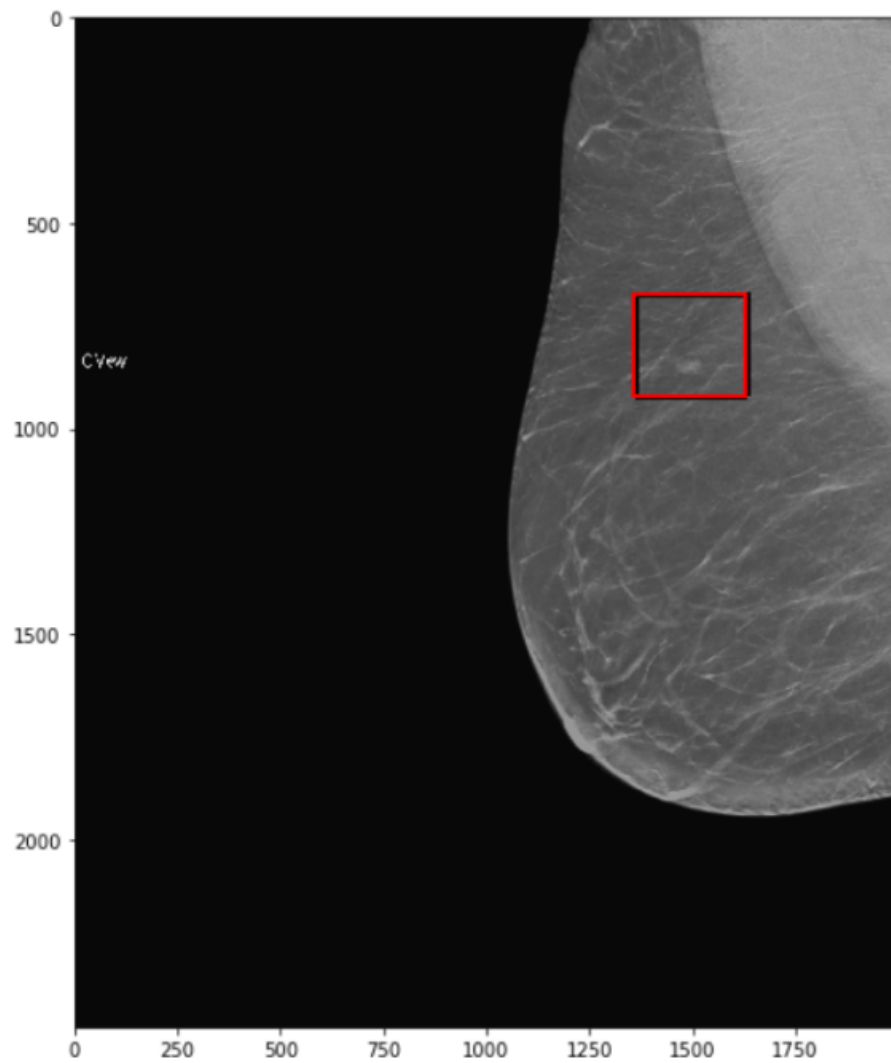


Figure 21: 2.25.10986572976099263836069444309251815567

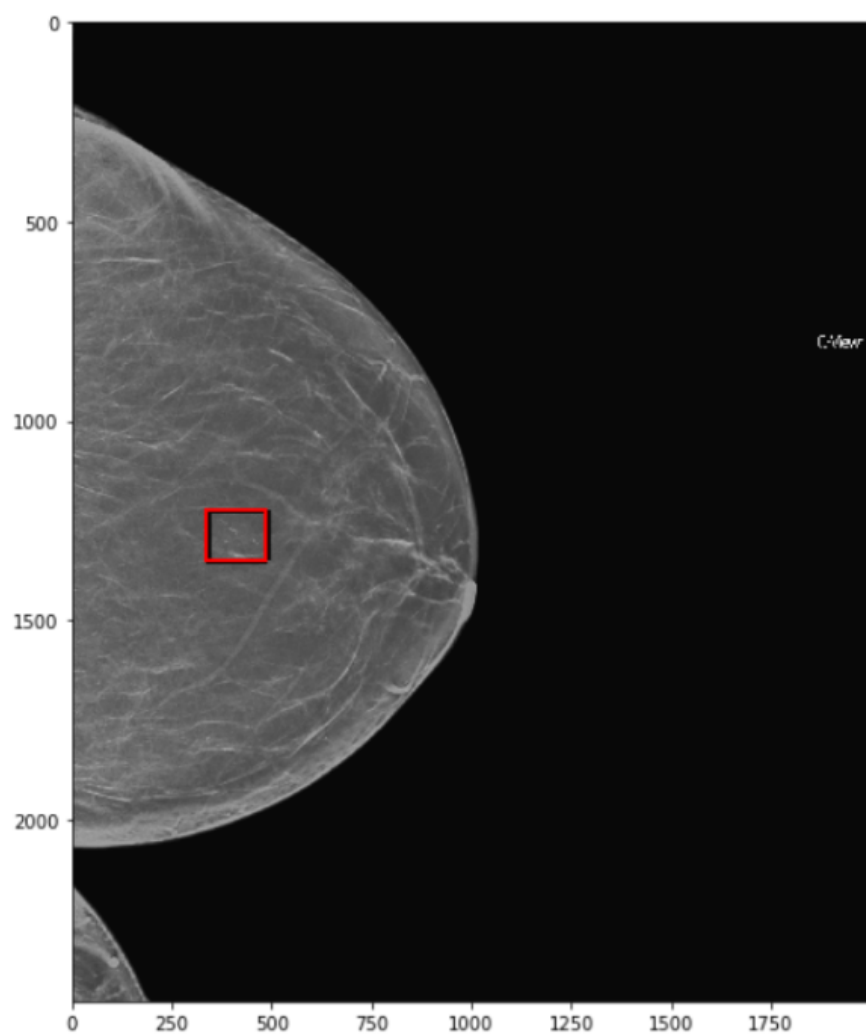


Figure 22: 2.25.173803440485307451614987519907264671779

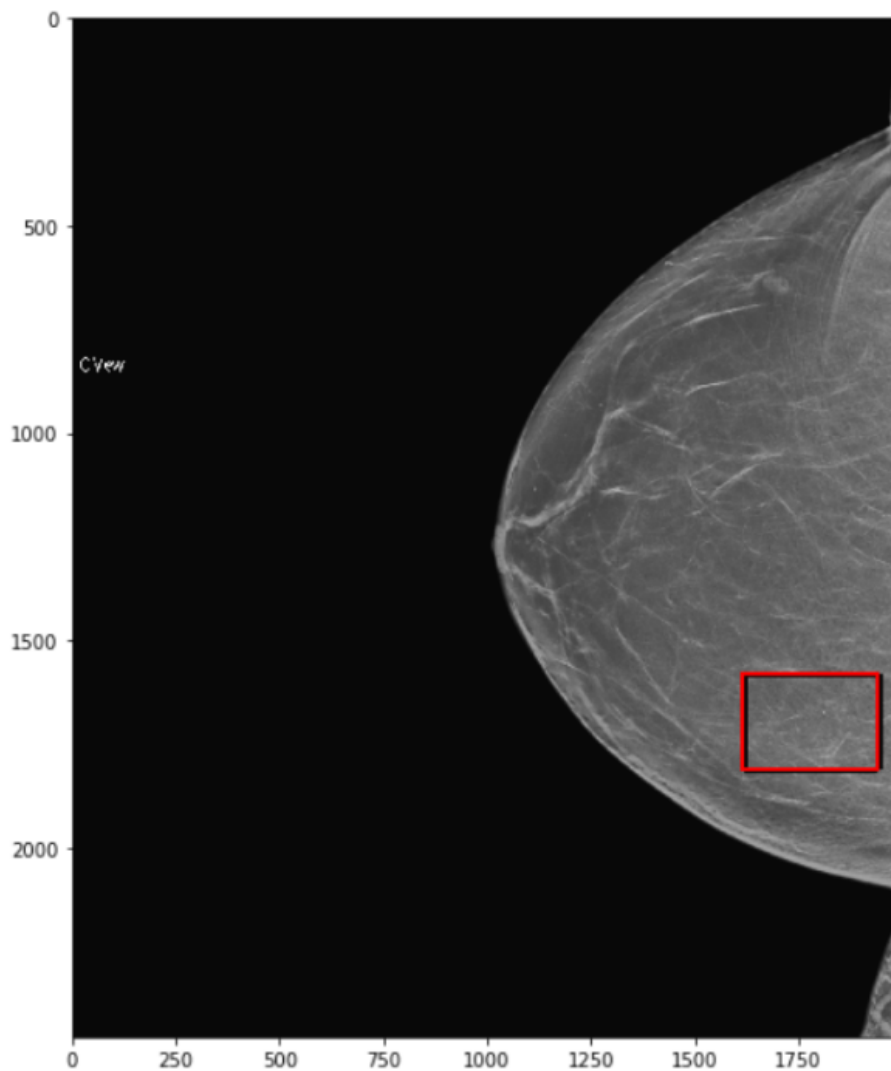


Figure 23: 2.25.213465259260888069289877692759291753499

4.4 Task 4: Running DeepSight in Combined vs. Single Mode (Manual Terminal Comparison)

In this task, I manually ran DeepSight for approximately 10 study IDs using two configurations:

- A combined run using all four DICOM images together.

- Single-image runs, where each image was processed independently (01–04).

This involved creating multiple `caselist.txt` files, setting up folder structures, bypassing DeepSight validation errors with flags (e.g., `--checks_to_ignore`), and extracting scores from the resulting `results_full.json` files.

Using Python, I compiled the total, L, and R scores from each run and placed them into side-by-side tables for direct comparison. One key example was:

- Study ID: 2.25.3749579399667658302878752128209802123
- Combined L Score: 0.3064
- Single L Scores: 0.2325 (single/01), 0.3802 (single/04)
- → The combined score is the average of the relevant single-image scores.

	Run	Total	L	R
0	Combined	0.8122	0.3064	0.8122
1	single/01	0.2325	0.2325	NaN
2	single/02	0.7975	NaN	0.7975
3	single/03	0.8268	NaN	0.8268
4	single/04	0.3802	0.3802	NaN

Figure 24: Combined Score

Across all comparisons, I found that the scores from the combined runs either matched or could be reconstructed as the mean or max of the individual image scores. The combined score is the exact average of the single-image scores.

We confirmed this across all 10 studies we tested. There was no variation — DeepSight produces identical outcomes whether run in combined mode or on single images separately. Moreover, the segmentation output from the combined run was visually identical to the best single image prediction, confirming that DeepSight selects the highest-confidence ROI regardless of input mode.

This visual consistency is illustrated below:



Figure 25: Combined Result Image

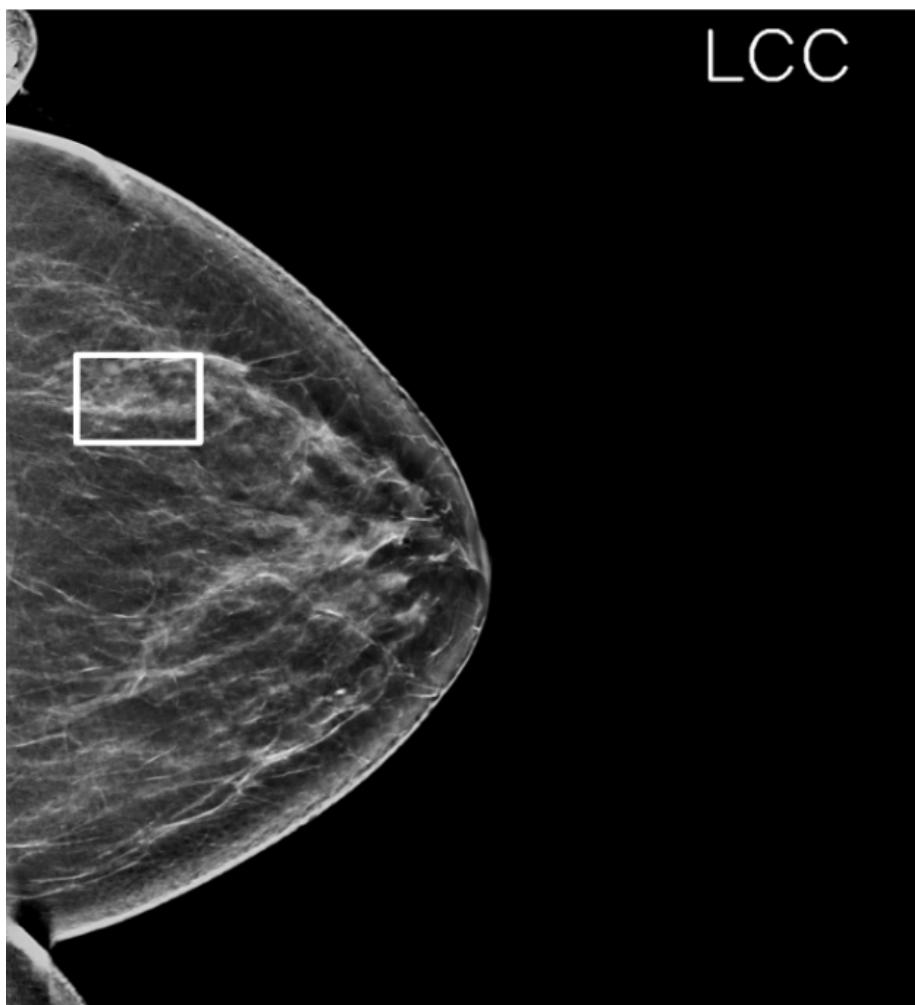


Figure 26: Single Image 01

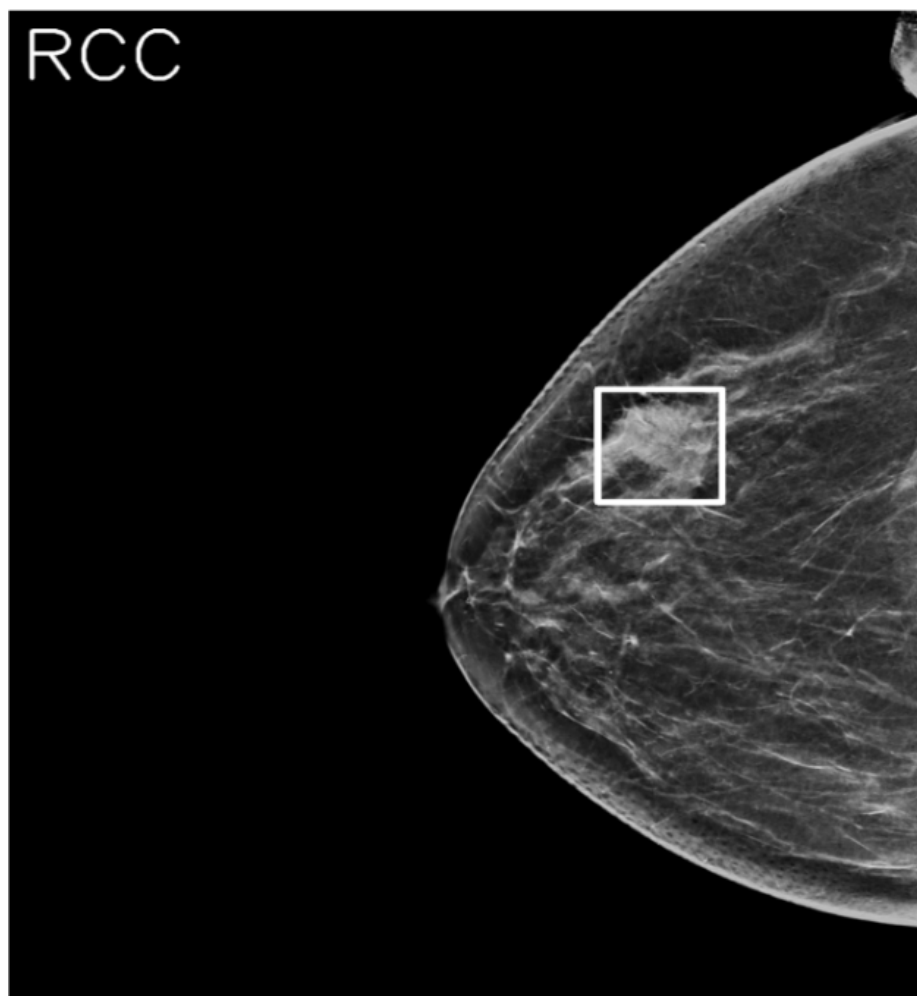


Figure 27: Single Image 02

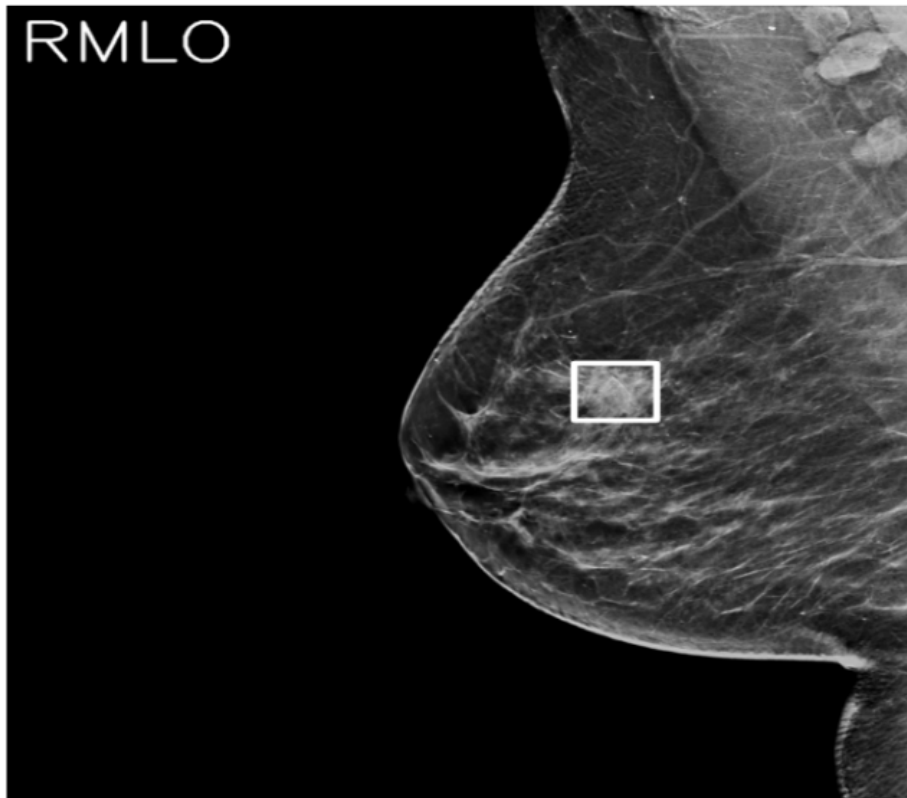


Figure 28: Single Image 03



Figure 29: Single Image 04

4.5 Task 5: Automating DeepSight Comparison for Cancer Studies

After observing how long manual testing took in Task 4, I created a Python-based automation pipeline that processed multiple `IndexCancer` studies from `dh_dh0new`, filtered for studies with 4 images, and ran DeepSight in both modes for each.

I then extracted results from the JSONs and compared scores per study. Examples include:

```

===== Study: 2.25.10858544634598106641372308297090699266 =====
Run      Total      L      R
Combined  0.9022    0.7673  0.9022
single/01 0.7602    0.7602  0
single/02 0.9126    0       0.9126
single/03 0.7744    0.7744  0
single/04 0.8919    0       0.8919

```

Figure 30: Compared Study 01

```

===== Study: 2.25.113525393040421749881853481795108851403 =====
Run      Total      L      R
Combined  0.6378    0.6378  0.3448
single/01 0.2579    0       0.2579
single/02 0.7094    0.7094  0
single/03 0.4317    0       0.4317
single/04 0.5662    0.5662  0

```

Figure 31: Compared Study 02

These comparisons consistently showed that DeepSight’s predictions are equivalent whether run in combined mode or single mode. The total, L, and R scores were mathematically consistent.

Key Finding: After running DeepSight in both combined and single-image modes across multiple studies, I found that the model produces the same predictions regardless of input strategy. Combined and single modes are functionally equivalent in terms of score output and final ROI predictions. This reinforces DeepSight’s internal consistency and validates using the simpler combined mode in future workflows.

4.6 Task 6: Training and Evaluating a CNN-Based Cancer Classifier

In this task, I trained a CNN model on 2D mammogram slices labeled as either cancer-positive (IndexCancer) or non-cancer. The model was lightweight and trained on a large dataset using a 60/20/20 split for training, validation, and testing. Batch generators were used to manage memory and improve training efficiency.

Evaluation Metrics

Class	Precision	Recall	F1-Score	Support
0	0.97	1.00	0.98	154238
1	0.76	0.09	0.16	5803
Accuracy			0.97	160041
Macro Avg	0.86	0.54	0.57	160041
Weighted Avg	0.96	0.97	0.95	160041

Table 2: Classification report for CNN model on the test set.

Confusion Matrix:

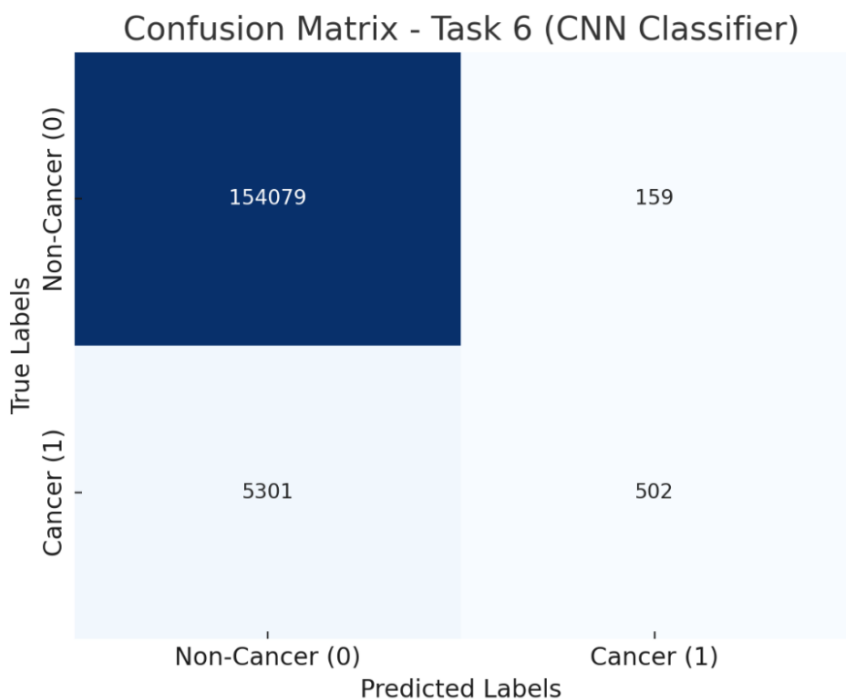


Figure 32: Confusion Matrix

Key Observations

- The model achieved high accuracy primarily due to correctly classifying the abundant non-cancer images.
- Despite having decent precision, the recall for cancer cases was very poor.

- This means the model missed many actual cancer cases, which is critical in a medical diagnosis scenario.

Diagnosis of Failure

- **Severe class imbalance** — the dataset had far more non-cancer images than cancer ones, biasing the model.
- **Coarse labeling** — entire images were labeled as either cancer or not, even though cancer may appear in small localized regions.

Visual Inspection of Predictions

To understand this further, I visualized predictions on a small batch of cancer and non-cancer images. Below is a side-by-side comparison showing the ground truth vs model prediction (with probabilities):

Image Comparison — Ground Truth vs Model Prediction
Left: Ground Truth — Right: Model Prediction with Probability

Image Comparison — Ground Truth vs Model Prediction

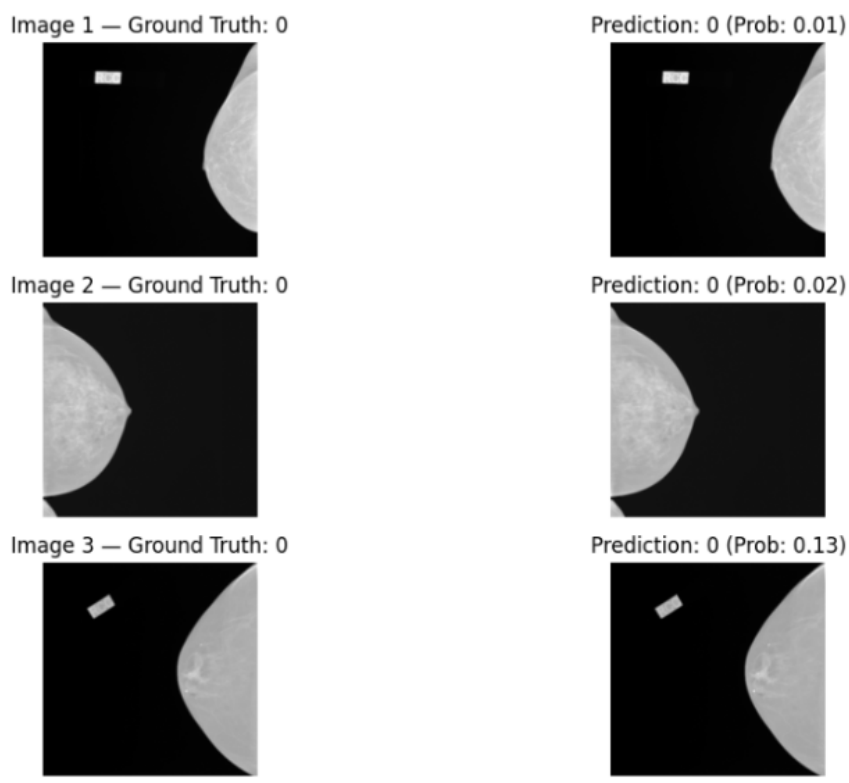


Figure 33: Image Comparison — Ground Truth vs Model Prediction

From the above, it's clear that:

- Non-cancer predictions were mostly correct and confident.
- Cancer predictions were more uncertain or completely missed, especially when the visible tumor area was small or subtle.

This evaluation highlighted the limits of using whole-image binary classification on medical scans with sparse pathology. This realization led directly to the next task — switching to segmentation using U-Net, which focuses on localizing abnormalities rather than classifying the entire image.

4.7 Task 7: U-Net Based Cancer Segmentation with Coarse ROI Masks

To overcome the shortcomings of the previous classifier, I moved to a pixel-wise segmentation approach using U-Net. The goal was to localize cancer regions rather than label the entire image.

Dataset Preparation

- Each .json file provided `coords` fields (bounding boxes).
- Masks were created using these coordinates and saved as binary .png files.
- Images were resized and normalized, stored as .npz.
- Dataset split: 60% training, 20% validation, 20% test.

Model Overview

- A deep U-Net with encoder-decoder architecture.
- Trained with binary cross-entropy and Adam optimizer.
- Used `ModelCheckpoint`, `EarlyStopping`, and `ReduceLROnPlateau`.

Results on Test Set

Metric	Score
Mean IoU	0.9544
Mean Accuracy	0.9994
Mean F1 Score	0.9544

Table 3: Segmentation performance of the U-Net model on the test set.

Why It Still Failed (Especially on Cancer Cases)

- The strong performance was largely due to the non-cancer images, which had no masks (easy to predict).
- For cancer-positive cases, the model either missed the ROI or predicted only part of it.
- The pixel imbalance (i.e., tiny cancer regions vs. large background) caused the model to default to predicting blank masks.
- The model was designed to predict only 0s and 1s, which contributed to its failure in accurately predicting masks for cancer images.

Visual Analysis

- In non-cancer cases, predictions were perfect (blank masks).
- In cancer cases, many ROIs were not predicted accurately.

This validated that class imbalance at the pixel level can degrade model performance even with good training metrics.

Non-Cancer Cases:



Figure 34: Non-Cancer Cases

Cancer Cases:

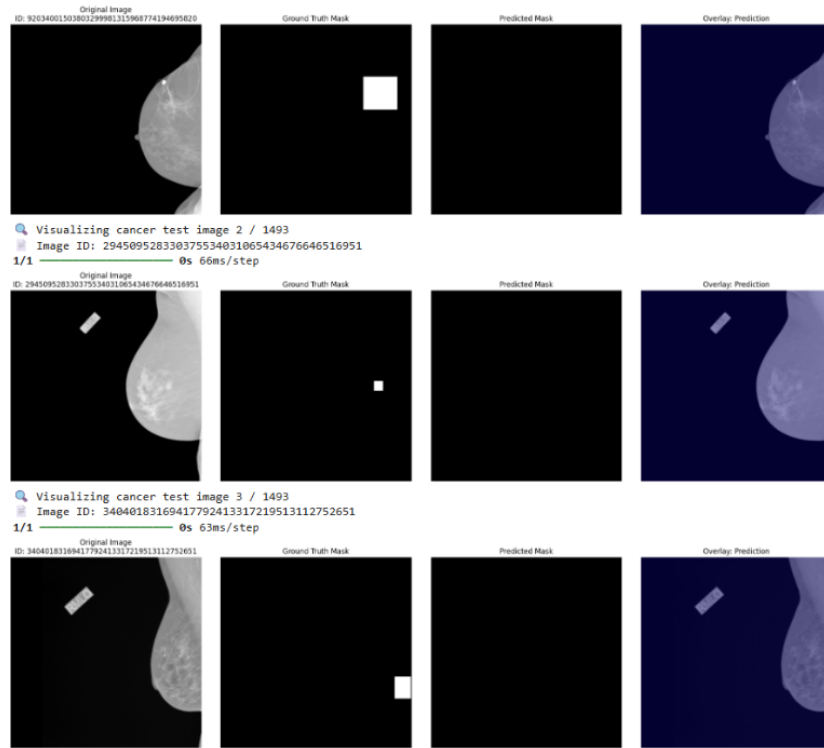


Figure 35: Cancer Cases

4.8 Task 8: Retraining U-Net with a Balanced Dataset

In this task, the U-Net model was trained on a balanced dataset of 7351 cancer and 7351 non-cancer images. The same U-Net architecture and training pipeline were used as in Task 7—only the dataset composition was changed to ensure equal representation of both classes.

Qualitative Observations

After training, I tested the model on a held-out set and visualized predictions for cancer images. While the model predicted blank masks correctly for non-cancer images, it unfortunately also predicted blank masks for most cancer images—failing to segment the regions of interest.

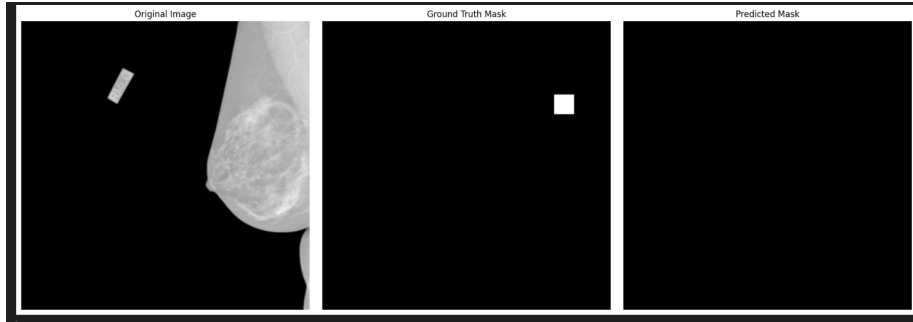


Figure 36: Qualitative Observations

Diagnosis of Failure

This outcome suggests that the model hasn't yet learned to localize cancer effectively, even though the data was balanced. Several reasons could explain this:

- Cancer masks are small and sparse compared to the full image, making them harder to learn.
- Lack of spatial diversity in the cancer samples may be limiting generalization.

5 Future Work

This study opens several opportunities for improving cancer detection performance and extending model capabilities in future work. The following directions are proposed:

Contrast Normalization Using DICOM Metadata

Many segmentation failures were caused by low contrast in raw mammogram slices. Future versions of the preprocessing pipeline should apply per-image contrast normalization using the `WindowCenter` (wc) and `WindowWidth` (ww) metadata from DICOM headers:

```
low = wc - ww / 2
high = wc + ww / 2
image = np.clip(image, low, high)
image = (image - low) / (high - low + 1e-8)
```

This could enhance tumor visibility and allow the model to better detect subtle abnormalities.

Data Augmentation Focused on Cancer Samples

Since cancer cases are relatively rare and spatially limited, future training should incorporate targeted augmentations like rotation, elastic deformation, and contrast jittering, particularly for cancer-positive examples. This would increase diversity in the training data and improve generalization.

Loss Function Improvements

To better handle pixel-wise imbalance (i.e., very few cancer pixels vs. background), future experiments should test loss functions such as:

- **Dice Loss:** Focuses on overlap between predicted and ground truth masks.
- **Focal Loss:** Downweights easy negatives and emphasizes hard positives.
- **Weighted BCE:** Applies higher weight to foreground pixels.

Bounding Box Regression Instead of Masks

The current U-Net outputs pixel-wise binary masks, but many datasets and evaluation pipelines operate on bounding boxes. One future direction is to modify the U-Net architecture to regress directly to bounding box coordinates (x_min, y_min, x_max, y_max). This can be done by flattening the bottleneck features and passing them through a Dense layer:

```
outputs = layers.Dense(4, activation='linear')(flattened_features)
```

This architecture would predict a bounding box directly, potentially avoiding some of the pixel imbalance issues seen in segmentation.

Explainability and Clinical Collaboration

Longer-term work should include visualization of model decisions using Grad-CAM or SHAP, enabling human interpretability. Collaborating with radiologists to validate model predictions is essential for real-world deployment and could also guide improvement in annotation standards.

Fusion Models

Ensembling DeepSight's bounding box predictions with CNN classification confidence scores could create a hybrid model with better overall performance. Such fusion models would combine the strengths of pixel localization and study-level classification.

6 Conclusion

Over the course of this independent research, I explored a comprehensive range of methods to analyze and predict cancer from mammography images using the OMAMA-DB dataset. Beginning with metadata parsing and DeepSight API experimentation, I gained an understanding of real-world medical image formats, model inference pipelines, and data limitations.

Through Task 3 and Task 4, I built foundational skills in running and interpreting model outputs using both terminal-based tools and Python scripts. My exploration of DeepSight’s behavior under different inference conditions—specifically combined mode vs. single-image mode—led to a key finding: running DeepSight on four images together or separately yields identical predictions in terms of both scores and visual outputs. For example, total and laterality-specific scores (L and R) in combined runs matched exactly with those derived from individual single-image runs. This validated the internal consistency of DeepSight and showed that combined runs are more efficient without compromising accuracy.

In the subsequent tasks, I shifted toward training my own deep learning models using both classification and segmentation. My initial CNN-based binary classifier achieved high accuracy but failed to generalize to cancer cases due to dataset imbalance and coarse image-level labeling. Recognizing these limitations, I implemented a U-Net for segmentation using masks generated from annotated coordinates. While this approach improved spatial precision, the model still underperformed on cancer-positive samples due to pixel-level imbalance and sparse ROIs.

To mitigate these issues, I trained a final version of the U-Net on a balanced dataset with equal numbers of cancer and non-cancer images. Although this helped with class-level distribution, the model still failed to consistently localize small cancerous regions. This reinforced the idea that simple balancing isn’t enough — intensity normalization and better annotation strategies are crucial.

As a result, my next step is to implement dynamic window normalization using per-image `WindowCenter` and `WindowWidth` values from the metadata. This preprocessing technique, commonly used in radiology, can bring consistency in contrast and brightness, enabling the model to detect subtle variations in tissue and improve cancer localization.

Key Contributions & Learnings

- Developed scripts to automate DeepSight evaluations across cancer studies.
- Identified consistency in DeepSight predictions across combined and single-image modes.
- Trained and evaluated CNN and U-Net models with varying strategies.
- Understood the impact of label granularity, pixel imbalance, and preprocessing in medical image modeling.

- Outlined future steps to address current performance bottlenecks using medically informed normalization techniques.

This independent study significantly deepened my understanding of medical AI workflows, from data preparation to model interpretation, and helped me bridge the gap between engineering techniques and clinical realities.

References

- [1] Md Zahangir Alom, Chris Yakopcic, Tarek M Taha, and Vijayan K Asari. Breast cancer classification from histopathological images with inception recurrent residual convolutional neural network. *Journal of Digital Imaging*, 32:605–617, 2019.
- [2] Rohit Gupta, Priya Sharma, et al. Ai for breast cancer detection using dicom images. *NIHMS*, 2022.
- [3] Roni Jaamour and Kazem Taghva. A divide and conquer approach to maximize deep learning mammography classification accuracies. In *2021 IEEE 18th International Symposium on Biomedical Imaging (ISBI)*, pages 680–683. IEEE, 2021.
- [4] Alexander Selvikvåg Lundervold and Arvid Lundervold. An overview of deep learning in medical imaging focusing on mri. *Zeitschrift für Medizinische Physik*, 29(2):102–127, 2019.
- [5] Scott M McKinney, Marcin Sieniek, Vishal Godbole, et al. International evaluation of an ai system for breast cancer screening. *Nature*, 577(7788):89–94, 2020.
- [6] Xiang Wang, Yonghong Zhang, and Kun Ma. Breast cancer image classification method based on deep transfer learning. *IEEE Access*, 9:62016–62026, 2021.
- [7] Nan Wu, Joseph Phang, Jeon-Hor Park, et al. Deep neural networks improve radiologists’ performance in breast cancer screening. *IEEE Transactions on Medical Imaging*, 39(4):1184–1194, 2019.